



Artificial Intelligence Software, Lecture 4

Model Selection and Sources



Center for
Cognitive
Modeling



Goal of the Lecture

Given a task: «**Find a model for Task X and run it**»

Our goal is to understand what a model is, what type of tasks exists, how to find it, what's important in it and how to launch it.



Architecture vs Weights

Architecture

- Defines the computation graph (layers, connections)
- Example: CNN, Transformer, MLP
- Same architecture can be reused across tasks

Weights

- Learned numerical parameters
- Stored after training
- Define the actual behavior of the model

Key idea: Same architecture + different weights = different models



What Does 2M / 85M Parameters Mean?

It's the number of learnable parameters.

Interpretation:

- 2M parameters → small model
- 85M parameters → larger capacity
- More parameters → more expressive, slower, heavier

The screenshot shows the GitHub repository for MAPF-GPT. The repository is owned by 'aandreychuk' and has a 'MIT' license. It contains a list of files and versions. The files are organized into a table with columns for the file name, size, and download links. The files include .gitattributes, LC-MAPF-3M.pt, MAPF-GPT-2M.pt, MAPF-GPT-6M.pt, MAPF-GPT-85M.pt, MAPF-GPT-DDG-2M.pt, README.md, model-2M-DDG.pt, model-2M.pt, model-6M.pt, and model-85M.pt. The sizes range from 1.52 kB to 1.02 GB. The download links are categorized into 'xet' and 'pickle' formats.

File Name	Size	Download Links
.gitattributes	1.52 kB	Safe
LC-MAPF-3M.pt	39.1 MB	Safe, pickle, xet
MAPF-GPT-2M.pt	19.1 MB	Safe, pickle, xet
MAPF-GPT-6M.pt	76.6 MB	Safe, pickle, xet
MAPF-GPT-85M.pt	1.02 GB	Safe, pickle, xet
MAPF-GPT-DDG-2M.pt	19.1 MB	Safe, pickle, xet
README.md	810 Bytes	Safe
model-2M-DDG.pt	19.1 MB	Safe, pickle, xet
model-2M.pt	19.1 MB	Safe, pickle, xet
model-6M.pt	76.6 MB	Safe, pickle, xet
model-85M.pt	1.02 GB	Safe, pickle, xet



How Can Models Share One Architecture?

- Architecture defines the structure (e.g. ResNet18)
- Parameters define the content inside this structure

Same architecture:

- ResNet18 (ImageNet)
- ResNet18 (medical images)

Differences:

- Different weights
- Different datasets
- Different performance



Why Parameter Count Changes

- Depth (number of layers)
- Width (number of neurons / channels)
- Embedding size (Transformers)

Example:

- ResNet18 vs ResNet50
- YOLOv8n vs YOLOv8m vs YOLOv8l

Effect:

- More parameters → better accuracy (usually)
- More memory and compute
- Slower inference



Unified Interface Across Models

- Despite differences, models share a common pattern:

`output = model(input)`

Examples:

- Image → classification scores
- Text → sentiment label
- Table → prediction

Important:

- Interface is the same
- Input format is NOT the same



Key Takeaways

- Architecture = structure of computation
- Parameters = learned knowledge
- 2M / 85M = number of parameters

Main insight:

- Same architecture can produce many models
- Difference is in weights and scale

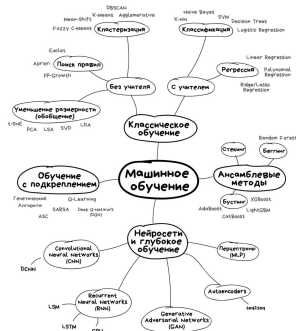
Practical view:

- Models look similar in code
- Real complexity is in data and preprocessing



Types of Tasks

- Computer Vision:
 - classification
 - detection
 - segmentation
- NLP (LLMs, embeddings)
- Tabular (classification/regression)
- Unsupervised (clustering)





Where to Find Models

- [Hugging Face](#) — model hub for NLP, CV, multimodal
- [GitHub](#) — code and model implementations
- [Kaggle](#) — notebooks and pretrained solutions
- [Torch Hub](#) — PyTorch pretrained models
- [arXiv](#) — research papers (often include models)
- [Ultralytics YOLO](#) — ready-to-use CV models for detection and segmentation
- [ClearML Model Registry](#) — examples and versioned models for MLOps



Cloud Environments for Running Models

- [Hugging Face Spaces](#) — deploy and run NLP, CV, and multimodal models in the cloud without local setup
- [Google Colab](#) — free cloud notebooks with CPU/GPU, ready to run Python, PyTorch, TensorFlow
- [Kaggle Kernels](#) — notebooks with preloaded datasets and pretrained models
- [Ultralytics YOLO HUB](#) — online demos and inference for detection/segmentation
- [ClearML Tasks](#) — run training, inference, and manage models remotely with MLOps tools



Model = Code + Weights + Docs

- Code: how to run
- Weights: learned parameters
- Docs: input/output format



How to Choose a Model

- Input format
- Output format
- Metrics
- Hardware requirements
- Latency / size



Preprocessing Matters

- Resize
- Normalize
- Tokenize
- Channel order (RGB/BGR)

Most errors come from wrong input



Examples

all material in extra_materials folder



Example: YOLO Detection

- Input: image
- Output: bounding boxes + labels
- Postprocess: threshold, NMS



Example: NLP Pipeline

- Input: text
- Output: label + score
- Tokenization inside pipeline



Classical ML Still Matters

- Logistic Regression
- Trees / Boosting
- Fast and simple baselines



MLOps: Model Management with ClearML

- **Store models** — upload trained models to ClearML Model Registry for central access
- **Model Details** — ready-to-use example projects showing model storage, retrieval, and deployment

Deserve "A" grade!

– Oleg Bulichev

✉ bulichev.ov@mipt.ru

📍 @Lupasic

🏠 Цифра, 521